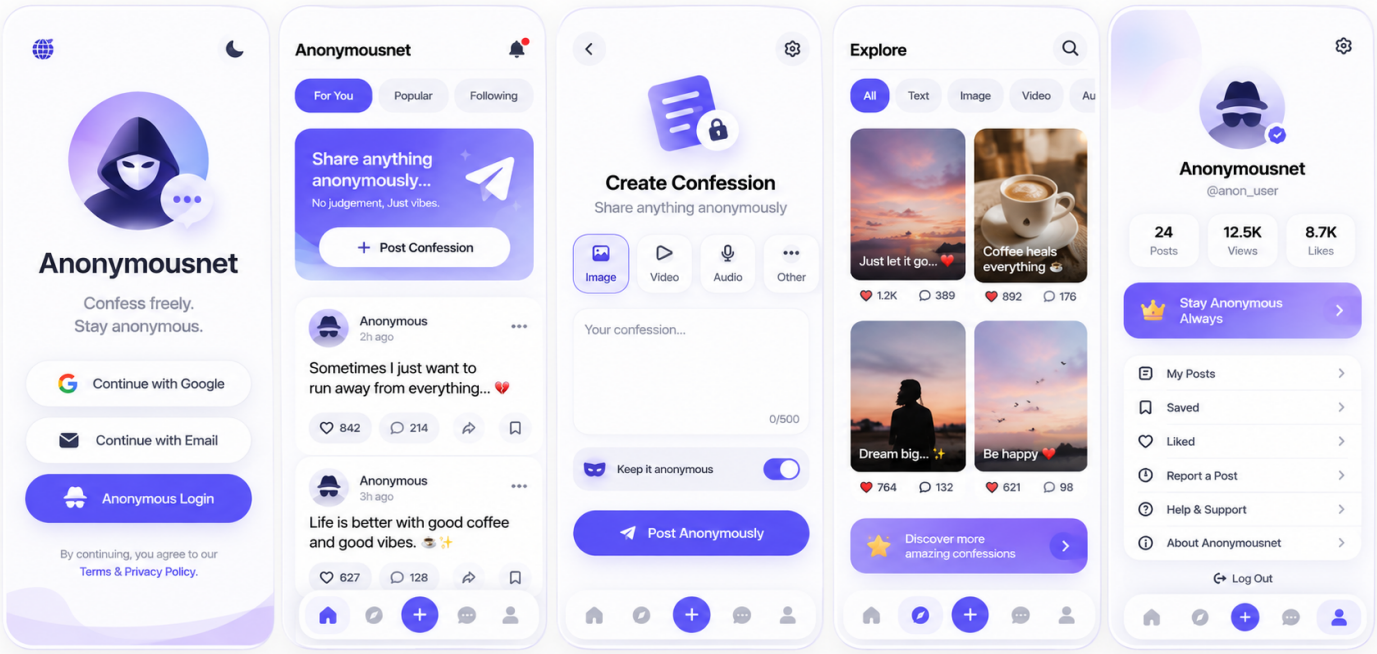
ANONYMOUSNET

Product requirements, UI/UX specification & implementation guide

A mobile-first anonymous social platform for confessions, thoughts, images, videos, audio and recorded moments.



Design direction: preserve the supplied five-screen reference — white space, soft lavender surfaces, vivid violet actions, rounded pill controls, gentle shadows and compact floating navigation.

Implementation baseline in this document: native Android app using Kotlin + Jetpack Compose, Firebase Authentication and Cloud Firestore/Storage. The architecture can be adapted to another client stack without changing the product or design requirements.

Document overview

This guide translates the supplied Anonymousnet mock-up into an implementation-ready product specification. It describes the exact visual system, screen behavior, data model, security rules, backend flows, media handling, testing and release criteria.

Item	Specification
Product	Anonymousnet — anonymous social media and confession sharing
Primary platform	Android, mobile-first; adaptive layouts for small and large phones
Client	Kotlin, Jetpack Compose, Material 3 components with a custom design system
Backend	Firebase Authentication, Cloud Firestore, Cloud Storage, Cloud Functions, FCM, App Check
Authentication	Google, email/password, and anonymous guest entry
Core content	Text, image, video, recorded audio/footage, and other supported attachments
Design source	The five-screen generated reference image supplied in this conversation
Product principle	Pseudonymous by default, safety-first, no public identity requirement

Important implementation note

Anonymous posting must not mean unmoderated or untraceable abuse. The app should avoid displaying a real identity, while still applying rate limits, reporting, content moderation, abuse controls and secure account recovery. Explain clearly what guest mode does and what data is retained.

Contents

- 1. Product vision and goals
- 2. UX principles and visual design system
- 3. Five-screen specification
- 4. Navigation and interaction rules
- 5. Functional requirements
- 6. Authentication and guest identity
- 7. Backend architecture and data model
- 8. Media upload and playback
- 9. Privacy, security and moderation
- 10. Notifications and real-time behavior
- 11. Accessibility, performance and adaptive UI
- 12. Analytics and operational monitoring
- 13. QA test plan and acceptance criteria
- 14. Build phases and release checklist

1. Product vision and goals

Anonymousnet lets people share honest thoughts without attaching their public identity. Users can publish short or long confessions, attach photos and videos, record audio, browse public posts, react, comment, save content and report harmful material. The experience should feel calm and approachable rather than chaotic or sensational.

Primary goals

- Make it possible to enter the app quickly using Google, email or anonymous guest access.
- Make creating a confession simple: choose a content type, compose, preview, and publish.
- Keep the author identity hidden on public content; show a consistent Anonymous identity/avatar.
- Offer a browsable feed and Explore discovery surface with filters for text, image, video and audio.
- Provide a profile area for a user's own activity, saved posts, liked posts, privacy and support.
- Make reporting, blocking, moderation and account deletion understandable and accessible.

Non-goals for the first release

- No direct messaging or public follower graph in MVP.
- No live streaming, public location tagging or contact-book discovery.
- No promise of absolute anonymity against network operators, platform providers or lawful requests.
- No arbitrary file execution or unrestricted attachment formats.

Success measures

Metric	What it indicates
Onboarding completion	Share of app launches that reach the feed or guest session
First-post conversion	New users who publish a first confession
Feed engagement	Meaningful likes, comments, saves and shares per active user
Safety response time	Time from report submission to moderation review/action
Media reliability	Upload success rate and playback start/error rate
Retention	D1/D7 returning users, interpreted alongside safety and quality metrics

2. UX principles and visual design system

The implementation must reproduce the reference's design language rather than invent a new visual direction. The five phones in the reference are the visual source of truth for proportions, spacing, surfaces, hierarchy and navigation.

Visual tokens

Token	Guideline
Canvas	Near-white background (#FFFFFF / warm white); keep generous negative space
Primary	Violet / indigo around #5950F2 for primary actions and selected states
Secondary surfaces	Very pale lavender (#F4F3FF) and cool off-white cards
Text	Near-black primary text; medium grey secondary text; avoid low-contrast grey
Corners	Large rounded cards; pill-shaped primary buttons and filter chips; circular icon buttons
Shadows	Soft, low-opacity shadows only; avoid heavy elevation and hard outlines

Token	Guideline
Typography	Modern sans-serif; bold screen titles; medium labels; readable body copy
Icons	Simple rounded line icons; consistent stroke weight; violet/grey/black states
Navigation	Floating rounded bottom bar with clear active pill/circle treatment
Motion	Short, subtle transitions; no excessive bounce, parallax or decorative motion

Layout rules

- Use a single-column phone layout with consistent horizontal insets of approximately 16–20 dp.
- Respect status bar, camera cutout, gesture navigation and system font scaling.
- Cards should stretch to available width; text wraps naturally without clipping.
- Keep primary actions reachable in the lower half of the screen where practical.
- Use the same bottom navigation geometry and icon order on Feed, Create, Explore and Profile.
- The login screen has no bottom navigation, matching the reference's focused onboarding screen.
- Support light mode as the reference default; offer dark mode with carefully tuned surfaces and contrast.

3. Five-screen specification

Screen 1 — Welcome and authentication

Purpose: explain the product and let a new user enter with minimal friction. Match the first reference panel: sparse white canvas, small brand mark, central anonymous/hooded illustration, centered title and three stacked rounded login actions.

- Top row: Anonymousnet globe/brand mark at left and a moon/theme control at right.
- Hero: anonymous avatar illustration centered in a soft violet circular glow.
- Title: “Anonymousnet”; supporting line: “Confess freely. Stay anonymous.”
- Actions, in order: Continue with Google; Continue with Email; Anonymous Login.
- Google button starts Google OAuth; Email opens sign-in/create-account form; Anonymous Login creates or resumes a Firebase anonymous session.
- Footer: concise Terms of Service and Privacy Policy links; disclose guest-account limitations.
- Show loading state and clear error feedback for each auth action. Never silently fail.

Screen 2 — Home / confession feed

Purpose: provide a calm, scrollable stream of anonymous posts. Match the second reference panel: compact header, notification bell, segmented feed tabs, violet create-confession banner, rounded post cards and floating bottom navigation.

- Header: Anonymousnet wordmark on left; notification bell with unread indicator on right.
- Tabs: For You, Popular, Following. In MVP, Following can be a saved topic/subscription feed if no social graph is implemented; otherwise label it “Topics”.
- Featured CTA card: “Share anything anonymously…” and “No judgment, just vibes.” with Post Confession action.
- Post card: anonymous avatar/name, relative timestamp, overflow menu, text/media, reaction row.
- Actions: like/unlike, comment, share via Android share sheet, save/unsave, report and block.
- Pagination: load in batches; show skeleton placeholders, empty states and retry states.
- Bottom bar: Home, Explore, Create (+), Activity/Inbox (if enabled), Profile. Keep icons and active-state styling consistent.

Screen 3 — Create confession

Purpose: make posting feel safe and straightforward. Match the third reference panel: centered title, content-type tiles, large rounded editor, anonymous toggle and violet publish button.

- Top row: back button at left; settings/draft options at right.
- Heading: “Create Confession”; subtitle: “Share anything anonymously”.
- Content type selector: Image, Video, Audio, Other. Text is the default editor mode.
- Composer: multi-line text input with a configurable character limit and live counter; support long text through a dedicated detail view if needed.
- Attachments: select from gallery or record audio/video using Android system permissions and capture APIs.
- Preview selected media before upload; allow remove/reorder where relevant.
- “Keep it anonymous” is ON by default and visually prominent. Do not expose author profile details in public post payloads.
- Publish button validates content, uploads media, writes the post, then returns to feed with a success confirmation.
- Save draft locally; drafts must remain private to the signed-in account/device.

Screen 4 — Explore

Purpose: discover public content by media type and popular topics. Match the fourth reference panel: title/search header, horizontal filter chips, two-column visual grid, compact engagement counts and violet discovery CTA.

- Header: Explore title and search icon. Search supports text, topic and safe content filters.
- Filter chips: All, Text, Image, Video, Audio. Add “Recent” and “Popular” sort controls where space allows.
- Grid: two-column cards with image/video thumbnails or styled text/audio previews; use consistent corner radius.
- Overlay short caption on media thumbnail; keep engagement counts beneath cards.
- Tap opens a full post detail view with comments and actions.
- Do not autoplay sound. Videos use muted preview or static thumbnail until opened.
- Exclude hidden, deleted, blocked, under-review and audience-restricted content from discovery.

Screen 5 — Profile and settings

Purpose: give users control over their anonymous activity and app preferences. Match the fifth reference panel: centered anonymous avatar, account label, small stats, highlighted privacy card and a clean settings list.

- Header: settings gear; anonymous avatar with optional verification/safety badge only if a real verification process exists.
- Identity: Anonymousnet display label and a non-identifying handle (e.g. @anon_user); do not show email publicly.
- Stats: posts, views and likes; label each metric accurately and avoid implying private analytics are public.
- Privacy card: “Stay Anonymous Always” opens privacy explanation and identity controls.
- Rows: My Posts, Saved, Liked, Report a Post, Help & Support, About Anonymousnet.
- Settings: theme, notification preferences, blocked users, content preferences, privacy policy, terms, account management, delete account and log out.
- Guest account: explain that uninstalling/clearing app data may lose access unless the guest account is linked.

4. Navigation and interaction rules

Element	Behavior
Bottom Home	Opens feed and restores feed scroll position
Bottom Explore	Opens discovery filters and grid
Center +	Opens Create Confession; preserve draft if user leaves

Element	Behavior
Activity icon	Opens notifications/activity list; unread count updates in real time
Profile	Opens profile, saved content and settings
Post card	Tap content opens detail; tap action icons performs only that action
Overflow menu	Report, block author, hide post, copy link; menu options depend on state
Back	Returns to previous screen; warn before discarding unsaved content

Use clear pressed, loading, success, disabled and error states. Avoid controls that look interactive but do nothing. Every action should have a defined backend result or be explicitly marked as unavailable in the MVP.

5. Functional requirements

Posts and content

- Create, read, edit (within a short configurable window), archive and delete the user's own posts.
- Supported MVP types: text, image, video and audio. “Other” should initially allow only an explicit allowlist such as PDF/document attachments if moderation and scanning are implemented.
- Posts can include caption, media metadata, content warning, topic tags, createdAt, status and engagement counters.
- One post may contain multiple images; video/audio attachment count and size limits must be configured server-side.
- Use server timestamps and stable IDs; never trust client-provided like/view counters.
- Implement cursor-based pagination and query indexes; do not download the entire feed.

Engagement and community

- Like/unlike is idempotent: one active like per user per post.
- Comments support create, edit own comment, delete own comment, report and block.
- Saved posts are private to the current account and never visible to other users.
- Share uses a public post deep link; the link must respect deleted/hidden/moderated post state.
- Block hides the blocked account's content and prevents interaction according to the block policy.
- Report flow requires a reason category and optional details; confirmation shows that the report was received.

Moderation and content states

State	Meaning / behavior
draft	Private, not visible in feeds
processing	Media upload or server validation in progress
published	Eligible for feed and Explore
under_review	Temporarily excluded from discovery pending review
removed	Unavailable publicly; retain only necessary audit metadata
deleted	User-initiated deletion; remove public content and schedule media cleanup

6. Authentication and guest identity

Firebase Authentication is the recommended baseline because the requested entry options map cleanly to Google sign-in, email/password and anonymous authentication.

- Google: use Credential Manager / Google Identity integration with Firebase Auth; do not embed OAuth client secrets in the app.
- Email: support create account, sign in, password reset, email verification and sign out.
- Anonymous: call Firebase anonymous sign-in and create a private user profile document with a random internal UID.
- Account linking: allow a guest to link Google or email credentials without losing posts, saved items or settings.
- Session persistence: use Firebase Auth state listener; restore the session on app relaunch.
- Account deletion: require re-authentication where Firebase requires it; delete user data and associated media under documented retention rules.

Important privacy distinction

The public interface should not expose a real name, email address, phone number or Firebase UID. However, Firebase still needs an internal account identifier to enforce ownership, prevent abuse and manage data. Do not advertise the service as “impossible to trace” or “100% anonymous”.

Suggested user-facing copy

“Your posts appear under an anonymous identity. We do not show your sign-in details on public posts. We still process limited account and technical data to keep Anonymousnet secure, prevent abuse and operate the service.”

7. Backend architecture and data model

Recommended architecture

Layer	Responsibility
Android app	Compose UI, navigation, local draft cache, Firebase SDK, media capture and playback
Firebase Auth	Google, email/password, anonymous sessions and credential linking
Cloud Firestore	Profiles, posts, comments, likes, saves, reports, blocks, notifications and settings
Cloud Storage	Original and processed image/video/audio objects with private upload rules
Cloud Functions	Trusted counters, moderation hooks, notification fan-out, media validation and cleanup
Firebase Cloud Messaging	Push notifications for relevant activity and moderation/account events
App Check	Reduce abuse from unauthorized clients; pair with rate limits and server validation

Firestore collection proposal

Path	Important fields
users/{uid}	createdAt, authProvider, accountState, settings, guest, linkedAt
publicProfiles/{uid}	displayName, anonymousHandle, avatarKey, createdAt; no email or real name
posts/{postId}	authorUid (private rules), anonymousLabel, text, media[], type, status, createdAt, updatedAt, likeCount, commentCount, viewCount, topics
posts/{postId}/comments/{commentId}	authorUid, anonymousLabel, text, createdAt, status
posts/{postId}/likes/{uid}	createdAt
users/{uid}/saved/{postId}	createdAt
users/{uid}/liked/{postId}	createdAt
reports/{reportId}	reporterUid, targetType, targetId, reason, details, createdAt, status
users/{uid}/blocks/{blockedUid}	createdAt

Path	Important fields
users/{uid}/notifications/{notificationId}	type, targetId, readAt, createdAt

Data modeling rules

- Keep private ownership data separate from public profile data.
- Use server-side Cloud Functions for counters, moderation status transitions and notification creation.
- Use Firestore transactions or batched writes for like/unlike and other multi-document invariants.
- Use server timestamps; validate text length, media type, size, owner and state.
- Create composite indexes for feed queries, Explore filters and user's own posts.
- Avoid storing large media blobs in Firestore; store only object paths, dimensions, duration, MIME type, size and checksum.

Security rules baseline

- Deny all access by default; grant only the minimum required operations.
- Only authenticated users may create content; anonymous Firebase users count as authenticated but receive tighter rate limits.
- Only the owner can edit/delete their own draft or post fields; clients cannot set trusted counters or moderation state.
- Public reads are limited to published posts and safe public fields; never expose authorUid in public post responses.
- Only a user can read/write their own saved, liked, blocked and notification subcollections.
- Reports are write-only for ordinary users; report contents are restricted to trusted moderation services/staff.
- Storage rules validate authenticated ownership, allowed MIME types and configured maximum sizes.

8. Media upload, capture and playback

Image flow

- Use Android Photo Picker for selected images; request broad gallery access only if a specific feature truly requires it.
- Generate a preview and optionally resize/compress before upload while preserving a safe original according to product policy.
- Strip EXIF GPS/location metadata by default; do not publish camera metadata.
- Upload to a per-user staging path, validate via trusted backend, then mark the post published.

Video and recorded footage flow

- Use Android Photo Picker for existing videos and CameraX for in-app recording.
- Show upload progress, retry, cancel and background upload status.
- Validate codec/container, duration, resolution and file size on the server; reject malformed or unsupported files.
- Generate poster thumbnails and transcode only when required for broad playback compatibility.
- Use adaptive playback or supported MP4/H.264 baseline for MVP; do not promise all codecs on all devices.
- Autoplay previews muted and only when visible; pause playback when a card leaves the viewport.

Audio flow

- Use Android audio recording APIs with explicit microphone permission and visible recording state.
- Provide duration indicator, waveform or simple progress, playback preview and delete/re-record controls.
- Never record in the background without clear user initiation and system-visible permission/indicator behavior.
- Store audio with MIME type, duration and ownership metadata; allow playback only for eligible published posts.

Upload reliability

- Use resumable Storage uploads; persist task state so transient network loss can recover.
- Do not mark a post published until every required media object is confirmed and validated.
- If upload fails, preserve the draft and allow retry without duplicate posts.
- Set explicit server-side size/duration limits and show them before users select large files.

9. Privacy, security and moderation

Anonymous social products need safety features at launch, not as a later polish task. Build abuse prevention into posting, comments, discovery and account management.

- Report categories: harassment, threats, self-harm concern, sexual exploitation, hate, spam/scam, impersonation, privacy exposure, illegal content and other.
- Provide block, hide, mute-topic and sensitive-content controls.
- Use automated text/media screening as a triage layer; high-risk content should be queued for human review where available.
- Rate-limit account creation, posts, comments, likes, reports and media uploads; apply stricter limits to guest accounts.
- Detect repeated spam, duplicate media, suspicious bursts and coordinated abuse without exposing private identity to other users.
- Create a documented escalation path for credible threats, child safety concerns and legally valid requests.
- Publish clear community guidelines, privacy policy, terms, retention schedule and contact route for safety issues.
- Support account deletion and content deletion with clear explanations of backups, legal retention and moderation records.

Privacy defaults

- Anonymous mode enabled by default for every new post.
- No public email, legal name, phone number, precise location or contact-list access.
- Location tagging disabled by default; strip geotags from uploaded media.
- No contact syncing in MVP.
- No advertising SDKs or third-party trackers unless disclosed and reviewed.
- Use TLS in transit, Firebase-managed encryption at rest and least-privilege service accounts.

10. Notifications and real-time behavior

- Use Firestore listeners for the active feed/detail/notification surfaces; detach listeners when screens leave scope.
- Use Cloud Functions to create notification records for comments/reactions where appropriate; avoid notifying users about every low-value event.
- FCM push notifications are opt-in and configurable by category.
- Notification types: comment on my post, reply to my comment, moderation result, account/security notice and product announcement.
- Do not expose sensitive confession text on lock-screen notifications by default; use generic copy such as “Someone replied to your post.”
- Maintain unread counts with trusted backend updates; mark read idempotently.

Real-time consistency

Optimistic UI may be used for likes and saves, but it must reconcile with the server result and roll back on failure. The UI must distinguish pending upload, published, failed and under-review states. Never show a success state before the backend confirms the action.

11. Accessibility, performance and adaptive UI

- Support Android font scaling, TalkBack labels, logical focus order and touch targets of at least 48 dp.
- Maintain accessible contrast for violet buttons, grey labels and selected/unselected icons.
- Provide captions or text alternatives for audio/video where feasible; do not rely on color alone.
- Use lazy lists/grids, image loading with thumbnail variants, caching and bounded Firestore queries.
- Compress thumbnails and decode images at display size; avoid loading full-resolution media in feed cards.
- Handle offline/poor connectivity with cached feed content, clear offline banners and retry controls.
- Adapt to different aspect ratios, cutouts and navigation modes; avoid fixed pixel positioning.
- Keep animations short and interruptible; respect system reduced-motion preferences.

12. Analytics and operational monitoring

- Track product events without recording confession text or sensitive media content in analytics.
- Suggested events: onboarding_started/completed, auth_method_selected, post_composer_opened, post_published, upload_failed, post_reported, post_saved, search_used and account_deleted.
- Use Crashlytics for crashes and non-fatal errors; Performance Monitoring for startup, screen rendering and network latency.
- Set alerts for upload failure spikes, function errors, auth failures, unusual report volume and moderation queue backlog.
- Define retention and access control for operational logs; do not log raw tokens, passwords, private message content or unnecessary identifiers.

13. QA test plan and acceptance criteria

Area	Acceptance criteria
Google login	A user can sign in, relaunch and retain the same account and content
Email login	Registration, verification, login, password reset and logout work end-to-end
Guest login	Guest session is created; posts are owned privately; linking preserves data
Feed	Only eligible published posts appear; pagination and refresh work without duplicates
Create post	Text/image/video/audio can be previewed and published; failure preserves draft
Engagement	Like, comment, save and share states persist correctly across relaunch
Explore	Filters return correct content types; deleted/blocked/under-review posts are excluded
Profile	Private lists are scoped to the user; email and UID are never public
Safety	Report and block flows work; blocked content is hidden; abuse limits are enforced
Media	Uploads resume/retry; thumbnails render; playback handles unsupported media gracefully
Accessibility	TalkBack, font scaling, contrast and touch targets pass review
Security	Firestore and Storage rules are tested with authenticated, guest and unauthenticated users

Required test matrix

- Test on low-, mid- and high-range Android devices and at least two Android OS versions supported by the release.
- Test Wi-Fi, mobile data, airplane mode, intermittent connection and upload interruption.

- Test empty feed, no search results, deleted post, suspended account, expired session and permission denial.
- Test large text, dark mode, landscape/rotation where supported, small screen sizes and display cutouts.
- Run Firebase Emulator Suite tests for Auth, Firestore rules, Storage rules and Cloud Functions.

14. Build phases and release checklist

Phase	Deliverables
1 — Foundation	Project setup, Compose design system, navigation, theme, Firebase environments, Auth providers
2 — Core social	Feed, post detail, create text confession, likes, comments, saves, user profile
3 — Media	Photo picker, image upload, video/audio capture and upload, thumbnails and playback
4 — Discovery	Explore grid, filters, search, sorting and pagination
5 — Safety	Report, block, moderation states, rate limits, privacy disclosures and account deletion
6 — Polish	Notifications, offline behavior, accessibility, analytics, performance and QA
7 — Release	Privacy policy, terms, store listing, data safety declaration, production rules and monitoring

Release checklist

- Replace all placeholder illustrations, logos and copy with approved production assets while preserving the mock-up layout.
- Verify Google OAuth SHA fingerprints, package name, Firebase app registration and release signing configuration.
- Deploy production Firestore/Storage rules and indexes; never ship permissive test rules.
- Configure App Check, quotas, billing alerts, Crashlytics and operational dashboards.
- Complete privacy policy, Terms of Service, Community Guidelines and support contact.
- Complete store disclosures for user-generated content, data collection, account deletion and permissions.
- Test report/block flows and moderation operations before public launch.

Final implementation direction

Build Anonymousnet as a real, production-oriented anonymous community product. Treat the supplied five-screen mock-up as the design contract: keep the white canvas, violet primary actions, rounded cards, soft lavender surfaces, spacious typography and floating bottom navigation consistent across the app. Add backend and safety capabilities without visually overcrowding the interface.

This document is a product and engineering specification. Production launch still requires implementation, security review, moderation operations, privacy/legal review and device testing.